

E0-243 HPCA Report:

Devashish Dhaulakhandi : 27231

November 2025

Contents

1	2D Convolution:	3
2	CUDA 2D convolution Methodology:	3
3	Transferring data across to the GPU:	3
4	Naive/Baseline:	5
4.1	NCU Reports:	5
4.1.1	Major Factors:	7
5	Variant 1: Global Memory Coalescing:	7
5.1	NCU Reports:	7
5.1.1	Major Factors:	8
6	Variant 2: Shared + Constant Memory:	9
6.1	NCU Reports:	9
6.1.1	Major Factors:	11
7	Variant 3: Register Tiling:	11
7.1	NCU Reports:	11
7.1.1	Major Factors:	12
8	Variant 4: Filter/Kernel Separation:	13
8.1	NCU Reports:	14
8.1.1	Major Factors:	16
9	Variants Comparison:	17

1 2D Convolution:

Convolution is a mathematical operation which describes a rule of how to combine two functions or pieces of information to form a third function. The feature map (or input data) and the kernel are combined to form a transformed feature map.

Typical implementations use a sliding-window operation where the kernel moves across the input image. For each placement of the kernel K on the input image A , it computes the dot product of the kernel and the overlapping image pixels.

2 CUDA 2D convolution Methodology:

In all the variants, the general pattern is followed:

- **Host to Device Transfer:** Copy input images and kernel weights from host memory to GPU global memory.
- **Thread Mapping:** Arrange threads in 2D blocks; each thread computes one or more output pixels.
- **Convolution:**
 - For each output pixel: load input elements (shared or registers)
 - Multiply by kernel weights and accumulate in registers
- **Write Back:** Store accumulated results to global memory with boundary checks.
- **Optimizations:**
 - Coalesced/global memory accesses using contiguous chunks
 - Shared-memory tiling for reuse
 - Register tiling for multiple outputs per thread

3 Transferring data across to the GPU:

In the given implementation, the data is transferred to the GPU from the Host Pageable-Memory, simply changing the same to Host-pinned-Memory, it improves the memory bandwidth slightly.

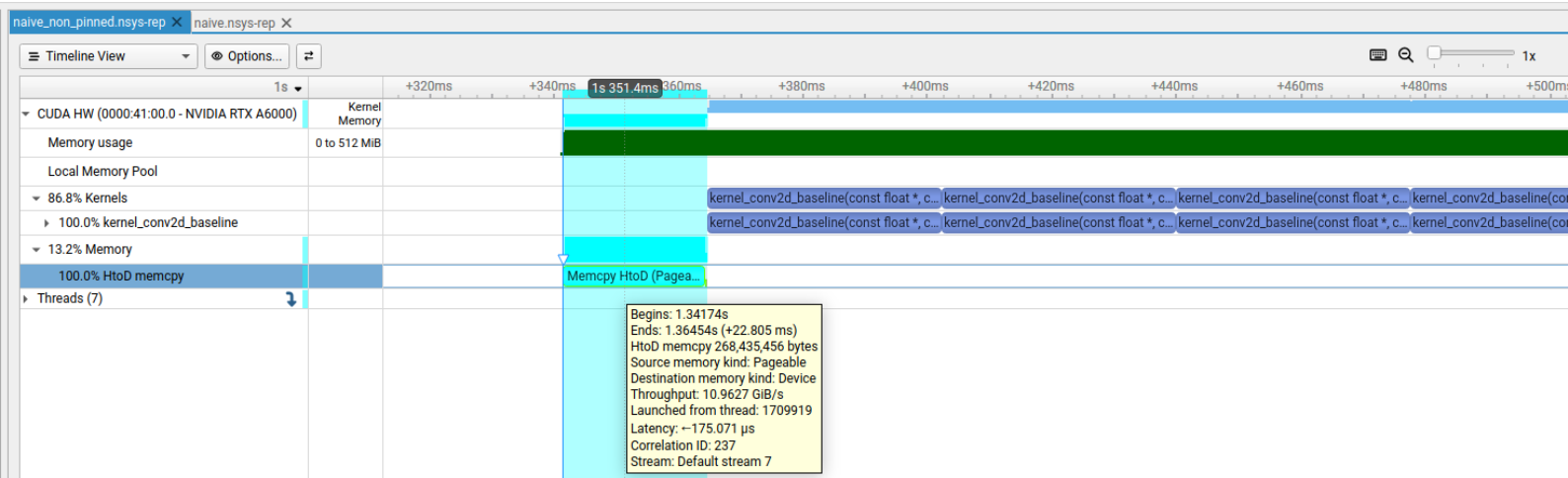


Figure 1: Using Pageable Host Memory (full-page width)

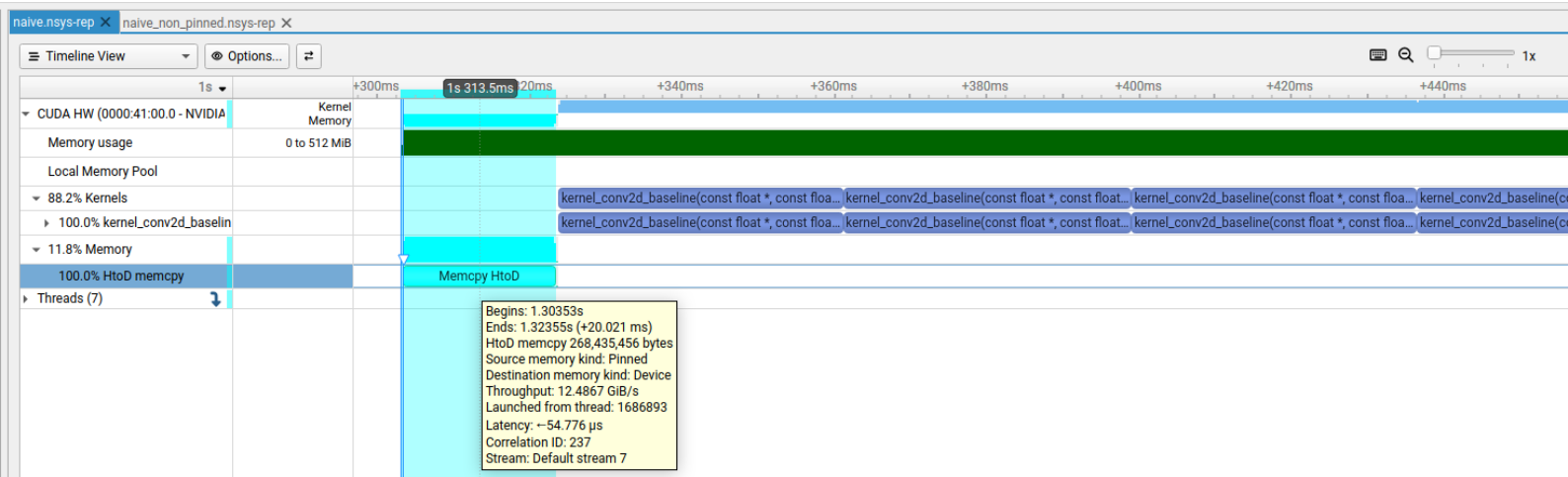


Figure 2: Using Pinned Host Memory (full-page width)

Memory Type	Size (Bytes)	Duration (ms)	Throughput (GiB/s)	Latency (μs)
Pageable	268,435,456	22.805	10.963	175.071
Pinned	268,435,456	20.021	12.487	54.776

Table 1: Comparison of HtoD memory copy for Pinned and Pageable host memory

The changes reduce time spent on the data transfer :

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Category	Operation
86.8%	150.365 ms	4	37.591 ms	37.591 ms	37.590 ms	37.593 ms	1.378 μs	CUDA_KERNEL	kernel_conv2d_baseline(const float *, const float *, float *, int, int, int, int)
13.2%	22.806 ms	2	11.403 ms	11.403 ms	992 ns	22.805 ms	16.125 ms	MEMORY_OPER	[CUDA memcpy Host-to-Device]

Figure 3: Pageable Memory Transfer (full-page width)

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Category	Operation
88.2%	150.370 ms	4	37.593 ms	37.592 ms	37.592 ms	37.593 ms	273 ns	CUDA_KERNEL	kernel_conv2d_baseline(const float *, const float *, float *, int, int, int, int)
11.8%	20.022 ms	2	10.011 ms	10.011 ms	992 ns	20.021 ms	14.156 ms	MEMORY_OPER	[CUDA memcpy Host-to-Device]

Figure 4: Pinned Memory Transfer (full-page width)

4 Naive/Baseline:

Metric	Value
Average Time (ms)	37.5981
Min Time (ms)	37.5951
Max Time (ms)	37.6013
Throughput (MPix/s)	1784.90
Ops per Pixel	121

Table 2: Run results for Naive/Baseline.

Section	Metric	Average
Compute Workload Analysis	Executed IPC	0.67 inst/cycle
Compute Workload Analysis	SM Busy	16.81 %
GPU Speed Of Light Throughput	Duration	37.59 ms
GPU Speed Of Light Throughput	Memory Throughput	14.22 GB/s
GPU Speed Of Light Throughput	DRAM Throughput	1.95 %
GPU Speed Of Light Throughput	L1/TEX Cache Throughput	97.84 %
GPU Speed Of Light Throughput	L2 Cache Hit Rate	86.56 %
GPU Speed Of Light Throughput	Compute (SM) Throughput	23.08 %
Occupancy per SM	Achieved Occupancy per SM	89.89 %
Launch Statistics	Registers per Thread	40 registers
Launch Statistics	Shared Memory per Block	8.19 KB
Launch Statistics	Waves per SM	520.13
Source Counters	Avg. Divergent Branches	487.62 branches

Table 3: Key average metrics for kernel

4.1 NCU Reports:

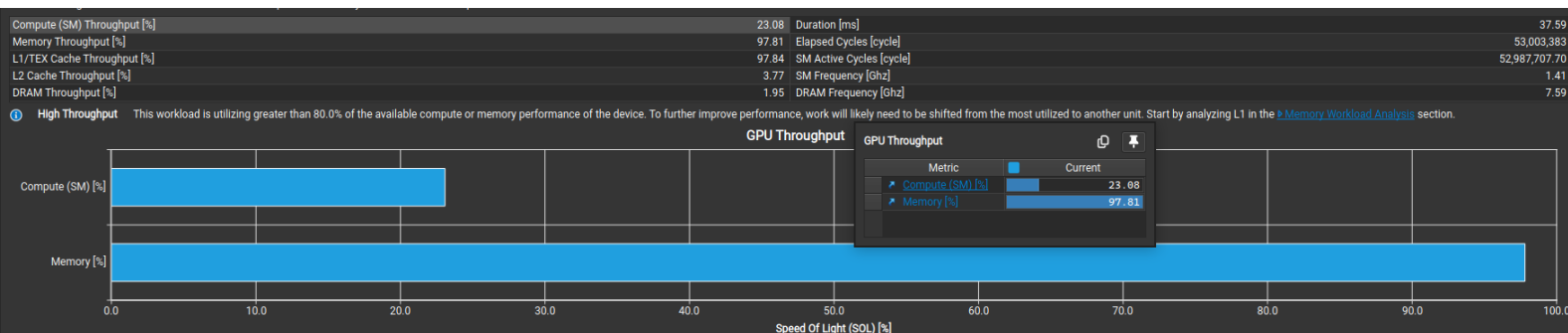


Figure 5: Naive

The Low Compute and High Memory usage shows that it is a Memory Bound Kernel. Poor coalescing affects the performance of the kernel.

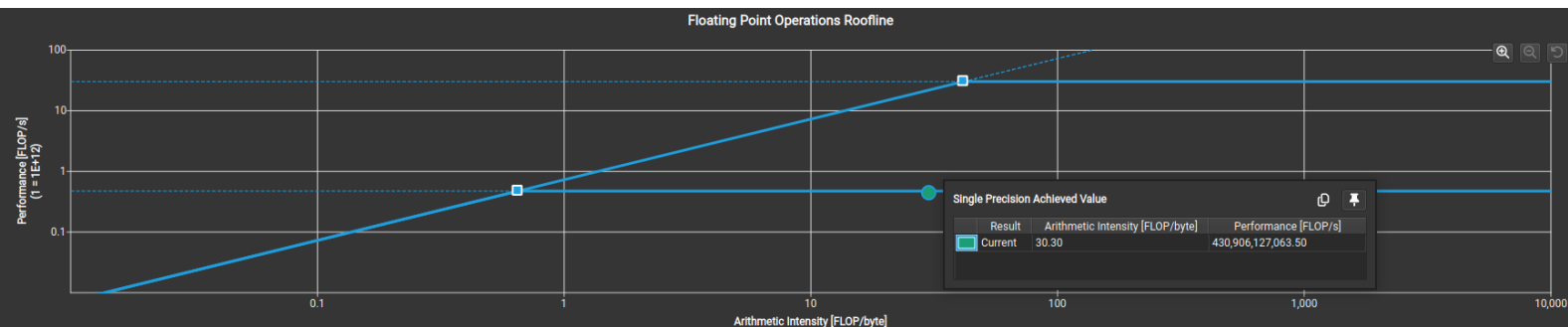


Figure 6: Naive

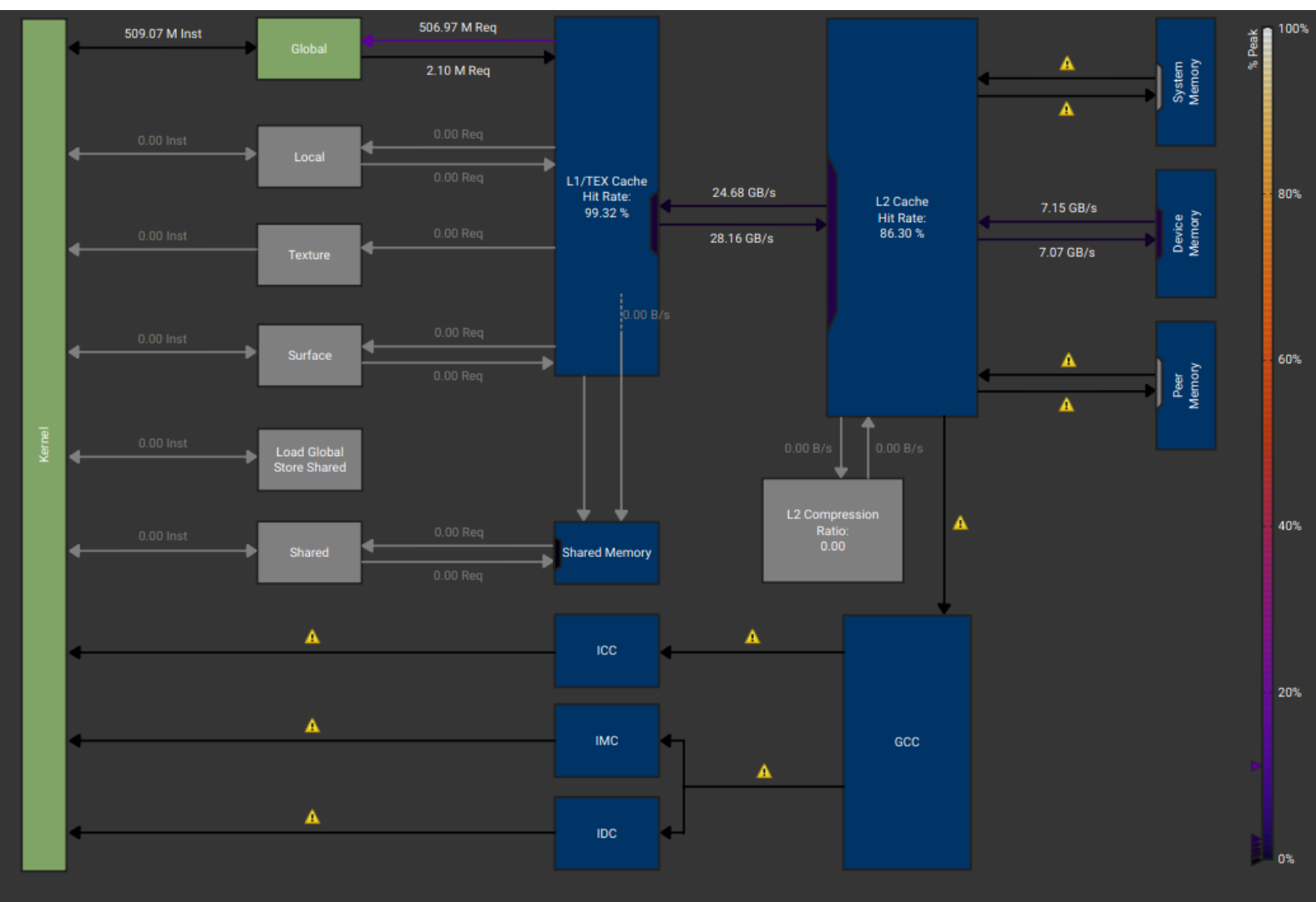


Figure 7: Naive

The L1 and L2 caches are being stressed/(a high fraction is used) but the runtime of the kernel is still very high and the throughput is low.

4.1.1 Major Factors:

The implementation is inefficient because of the threads per block dims (16,16,1), the rapidly changing threadID.X causes poor coalescing when threads of the same warp access diff. regions of the global memory.

Using threadIdx.x for row and threadIdx.y for column reduces performance because now adjacent threads have to access global memory at an offset (strided).

Due to these, there is poor throughput of data across the memory and the runtime is high.

5 Variant 1: Global Memory Coalescing:

Metric	Value
Average Time (ms)	9.0484
Min Time (ms)	9.0470
Max Time (ms)	9.0501
Throughput (MPix/s)	7416.68
Ops per Pixel	121

Table 4: Run results for Variant 1: Global Memory Coalescing.

5.1 NCU Reports:

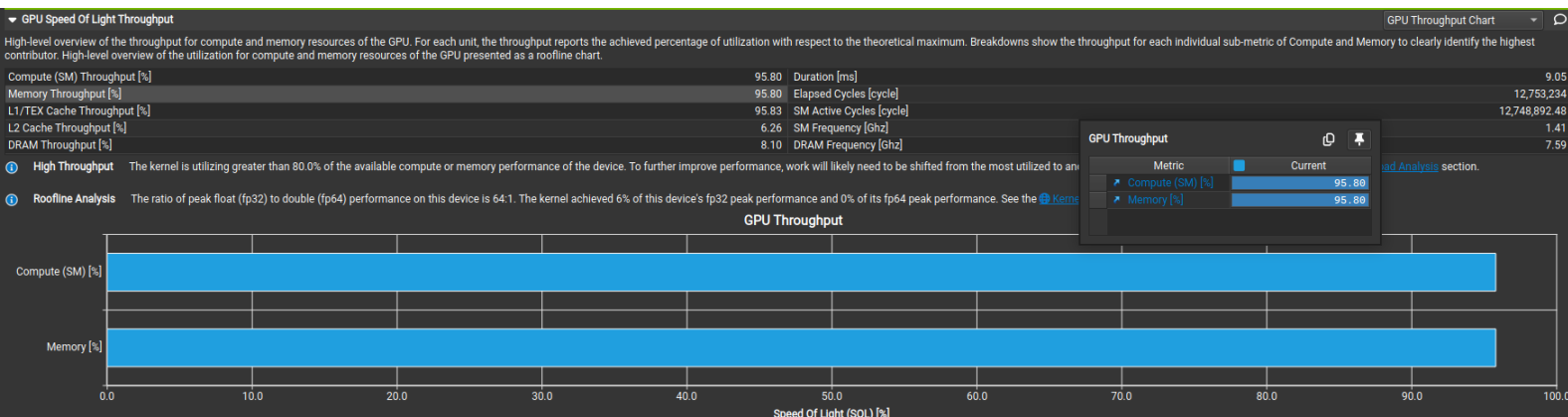


Figure 8: Variant 1

By changing the access pattern to global memory (using threads per block dim (32,8,1)) and each thread in the warp accessing / writing to adjacent output pixels, the runtime decreases significantly and the compute and memory usage both rise.

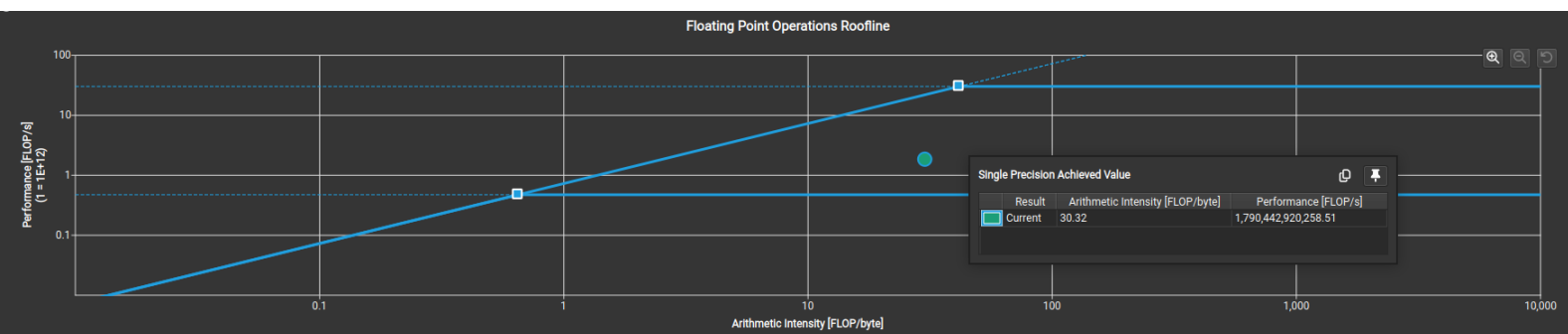


Figure 9: Variant 1

We see throughput is increased now and FLOPs/s is better than the naive version.

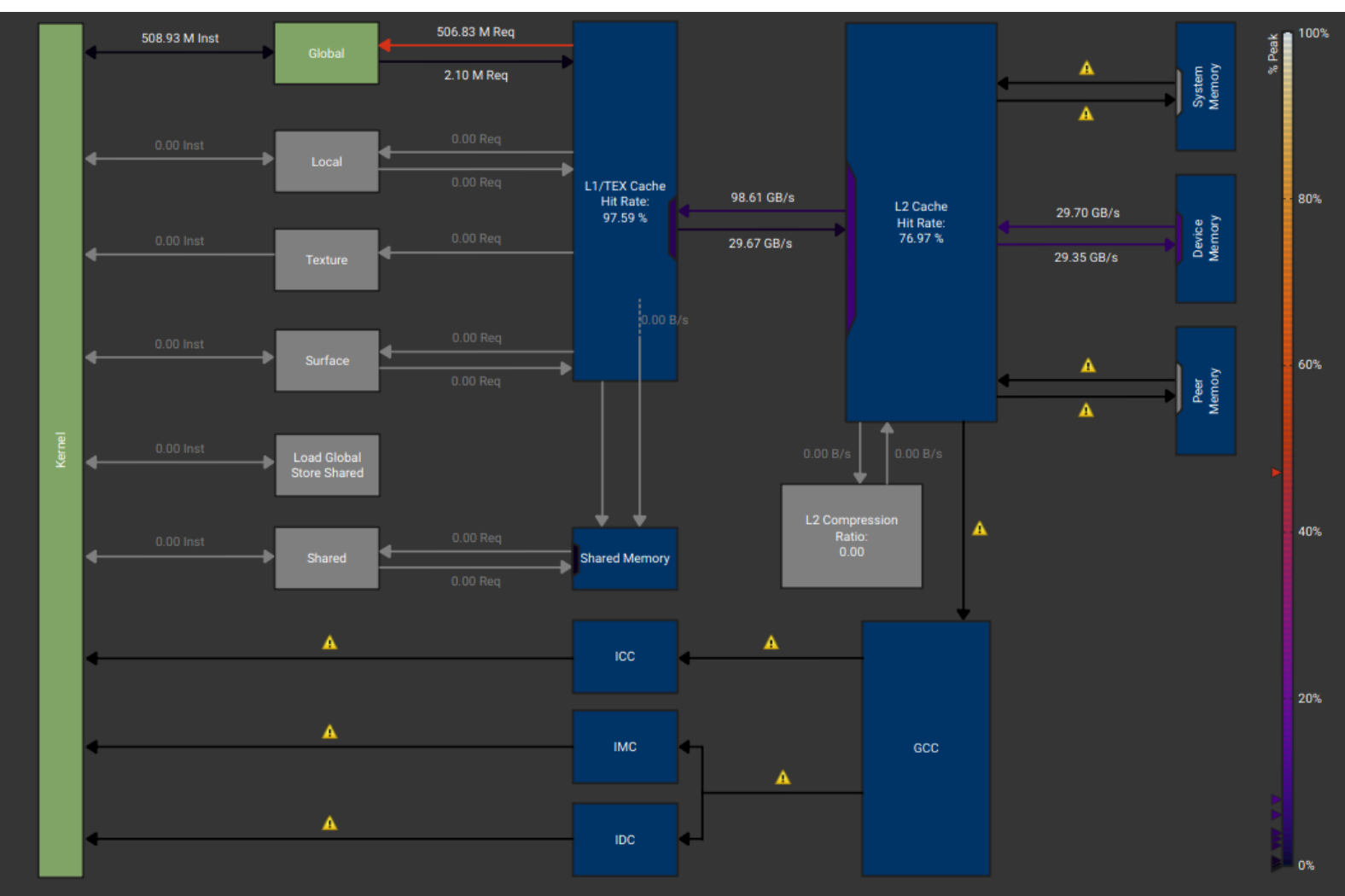


Figure 10: Variant 1

5.1.1 Major Factors:

Some of the previously strided accesses are now coalesced by improving the mapping between thread layout and the physical memory layout. As a result, memory

throughput increases noticeably and the GPU spends less time stalled on scattered global loads.

However, because the filter size is variable and alignment constraints are not guaranteed, 16-byte vectorized loads are not used, which limits how far coalescing can be pushed.

With more contiguous reads per warp, the L1 cache becomes heavily stressed and frequently accessed, while L2 also sees higher traffic. The compute units are active a large fraction of the time, so the bottleneck shifts from arithmetic to the global memory system and the caches themselves. Although the improvement is significant, further gains require reducing dependence on L1/L2 by using explicit data tiling in shared memory rather than relying on the hardware caches to handle reuse.

6 Variant 2: Shared + Constant Memory:

Metric	Value
Average Time (ms)	4.8106
Min Time (ms)	4.8097
Max Time (ms)	4.8128
Throughput (MPix/s)	13950.06
Ops per Pixel	121

Table 5: Run results for Variant 2: Shared + Constant Memory.

6.1 NCU Reports:

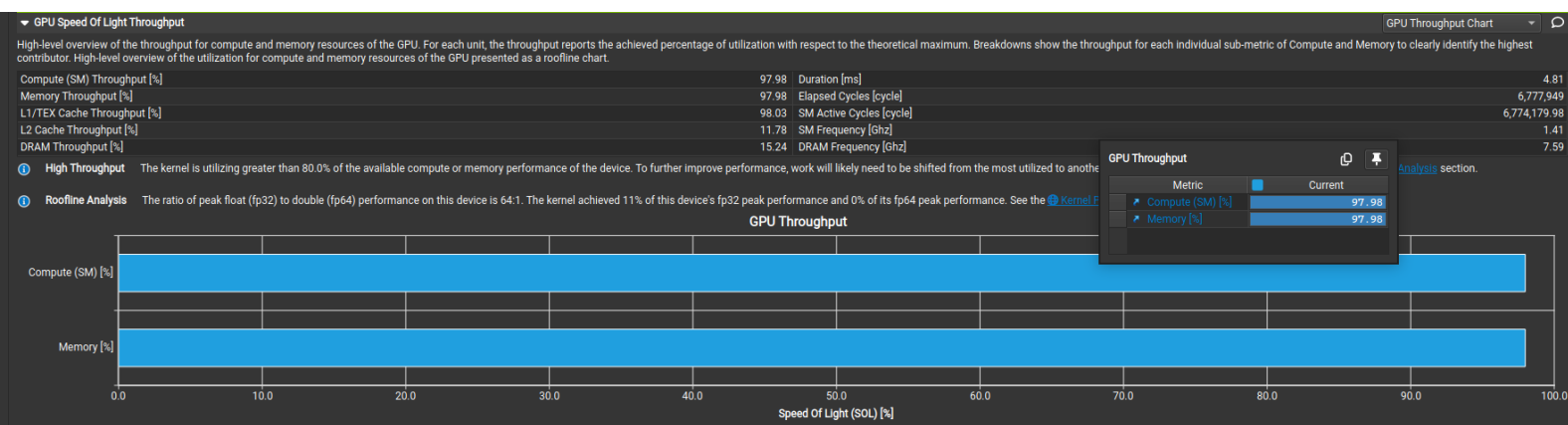


Figure 11: Variant 2

Using the shared and constant memory, the runtime is reduced even more since now data doesn't need to be accessed from the global memory as frequently and the compute is highly used along with the memory, a better balance is now maintained.

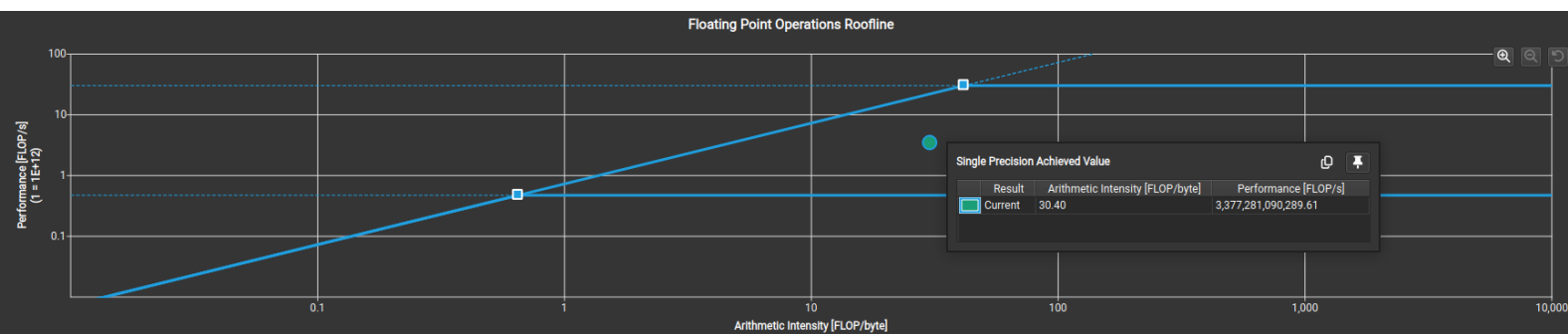


Figure 12: Variant 2

The FLOps/s increased.

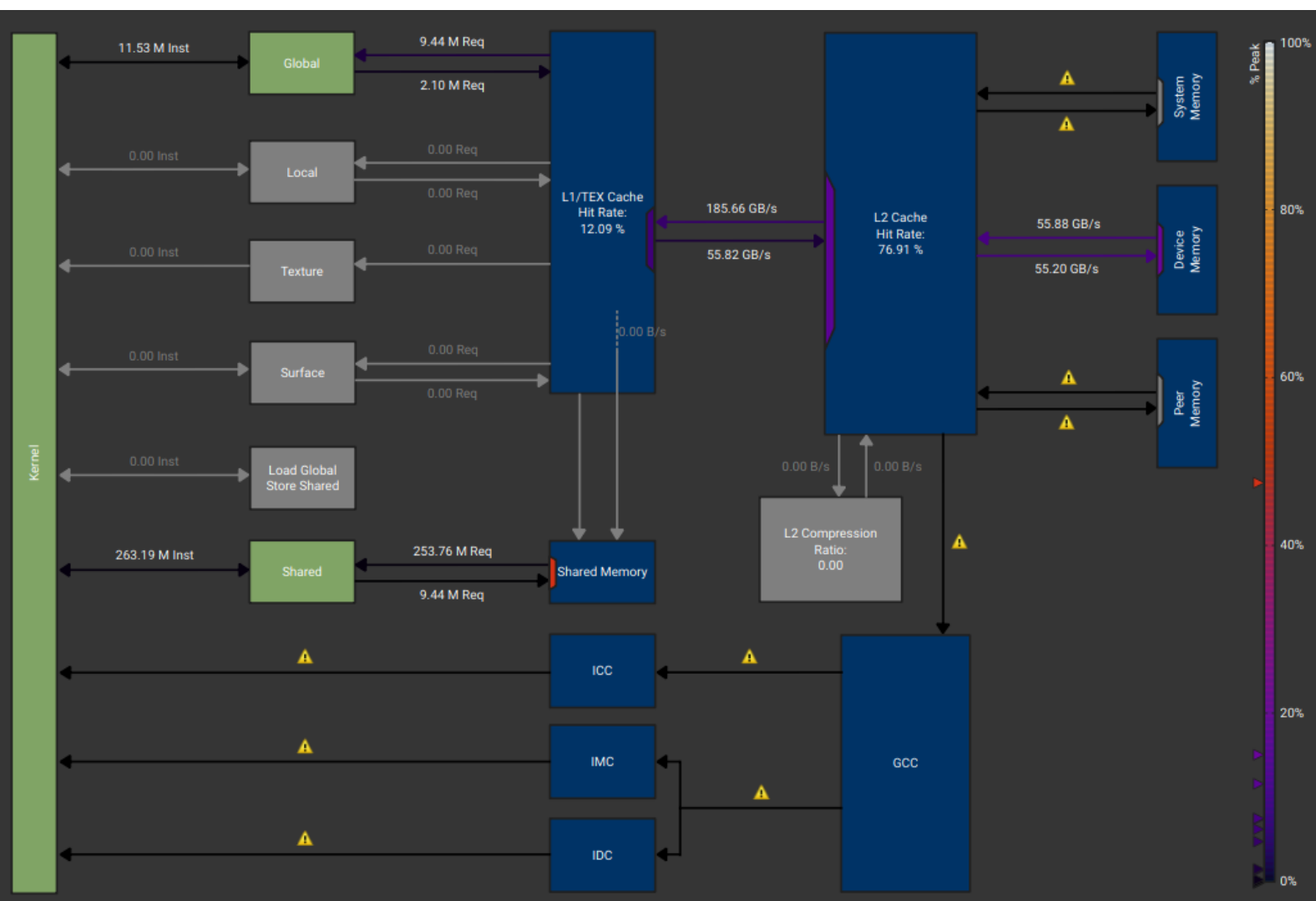


Figure 13: Variant 2

L1 cache is not stressed now since the shared memory (software managed) is used to access the input data. A large fraction of L2 is still in use. Device memory throughput is increased since compute is able to stress it now.

6.1.1 Major Factors:

The accesses to global memory are still not fully coalesced, since the access pattern has remained the same. This variant loads a tile of the input into shared memory and the kernel into constant memory, so most reuse now happens on-chip instead of repeatedly going back to global memory. Interestingly, profiler metrics show that DRAM bandwidth utilization increases while cache hit rates drop. This happens because the kernel now performs larger, more efficient streaming loads into shared memory and then reuses that data locally, reducing latency stalls even though fewer accesses hit L1/L2. The shared memory still suffers from some unavoidable bank conflicts, but the overall latency is greatly reduced because the GPU spends more time doing arithmetic and less time waiting on memory.

7 Variant 3: Register Tiling:

Metric	Value
Average Time (ms)	2.0505
Min Time (ms)	2.0490
Max Time (ms)	2.0521
Throughput (MPix/s)	32728.72
Ops per Pixel	121

Table 6: Run results for Variant 3: Register Tiling.

7.1 NCU Reports:

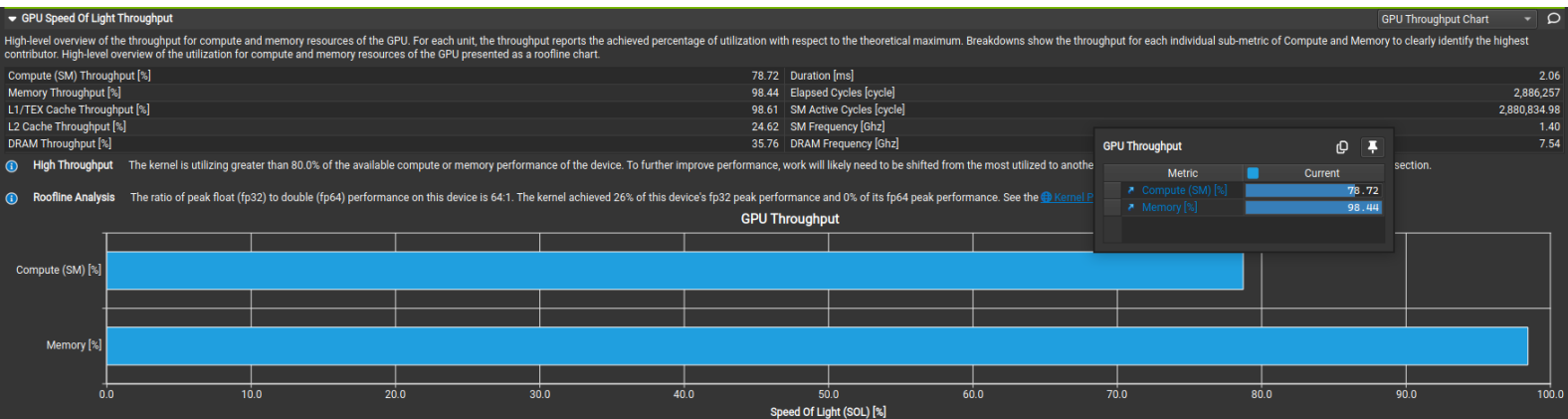


Figure 14: Variant 3

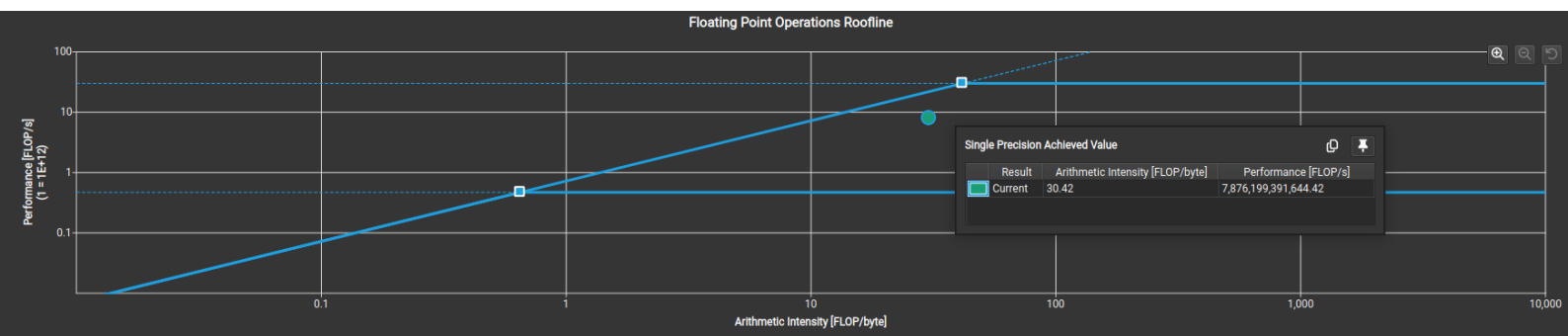


Figure 15: Variant 3

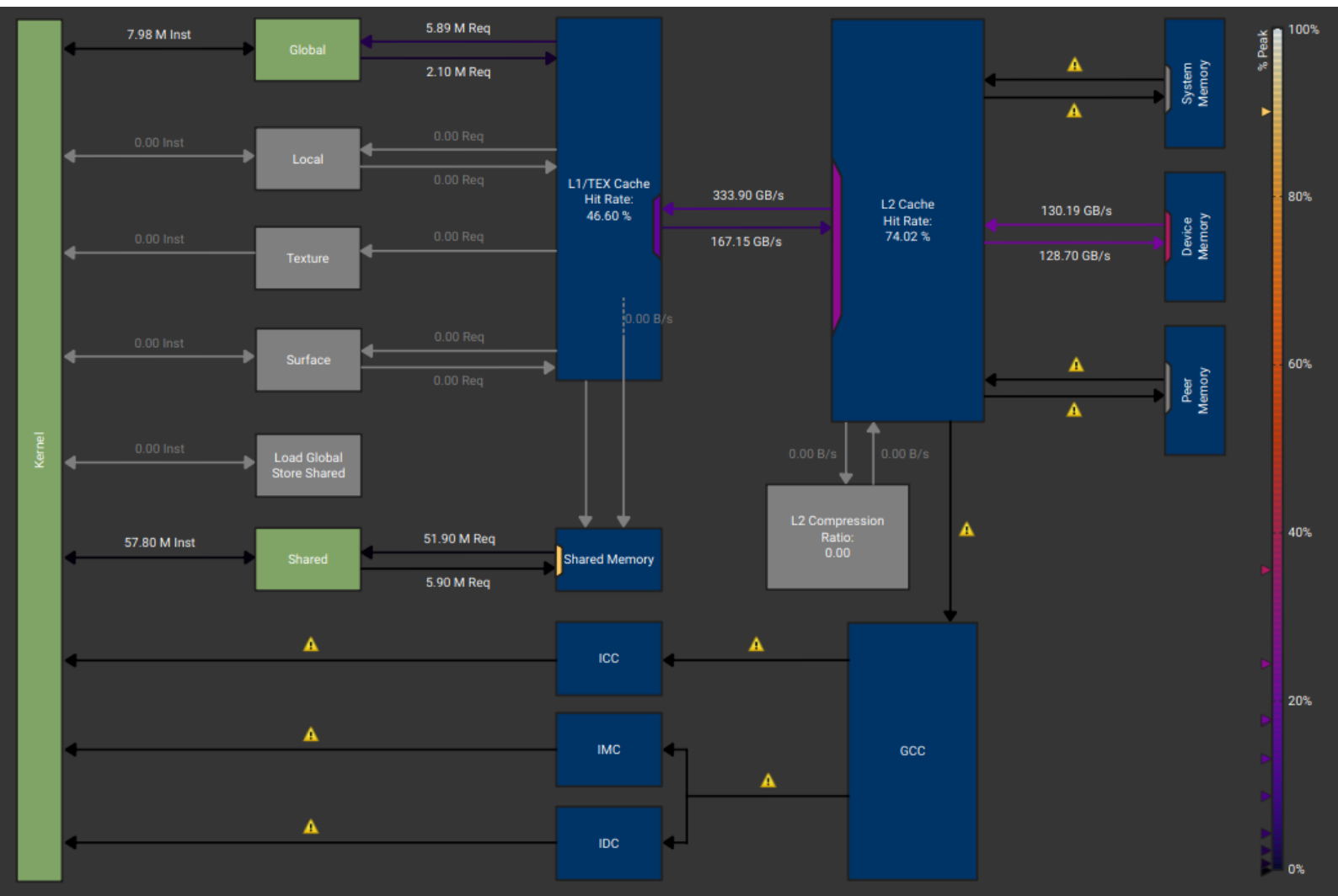


Figure 16: Variant 3

7.1.1 Major Factors:

Each thread computes multiple neighboring output pixels so that the overlapping input region is loaded once and reused directly from registers instead of repeatedly fetched from shared or global memory. The kernel exploits the fact that adjacent output pixels differ by only one column of input data, enabling efficient register tiling where each

thread produces four outputs from the same loaded data. This dramatically increases locality within the register file, raising instruction-level parallelism and reducing memory pressure across the hierarchy. As a consequence, L1, shared memory, and L2 all remain heavily utilized, and global bandwidth usage increases because more arithmetic progress is sustained per data fetched. Despite higher register usage slightly reducing occupancy, the much faster on-chip reuse dominates: runtime falls significantly since far fewer stalls occur on memory accesses and the SMs spend more time performing actual computation.

8 Variant 4: Filter/Kernel Separation:

A 2D convolution kernel $K \in R^{k \times k}$ is *separable* if it can be written as an outer product of two 1D vectors:

$$K = \mathbf{v} \mathbf{h}^\top, \quad \mathbf{h}, \mathbf{v} \in R^k.$$

This allows the 2D convolution

$$I_{\text{out}} = I_{\text{in}} * K$$

to be replaced by two 1D convolutions:

$$I' = I_{\text{in}} * \mathbf{h}, \quad I_{\text{out}} = I' * \mathbf{v}.$$

Thus, the cost per output pixel decreases from $\mathcal{O}(k^2)$ to $\mathcal{O}(2k)$, i.e., linear in the kernel size instead of quadratic.

Detection and Factorization. On the host, the kernel matrix is tested for rank-1 structure using SVD. If the second singular value is negligible compared to the first, the kernel is considered separable and is decomposed into a row vector $\mathbf{h}$ and a column vector $\mathbf{v}$.

Implementation on the GPU. When separability is detected, the program:

1. computes $\mathbf{h}$ and $\mathbf{v}$ on the CPU,
2. copies them to `--constant--` memory for fast broadcast access,
3. launches a horizontal 1D convolution kernel producing an intermediate buffer, and
4. launches a vertical 1D kernel to produce the final output.

Both passes benefit from the existing shared-memory tiling and register reuse strategies developed in earlier variants.

Performance Effects. Arithmetic workload and DRAM traffic are significantly reduced since each input pixel participates in far fewer multiplications. The intermediate buffer introduces another global write/read, but the reduction in total work dominates, improving overall throughput. Constant memory enables low-latency reads of filter coefficients, further minimizing bandwidth pressure.

$$\text{2D: } \mathcal{O}(k^2) \text{ ops/pixel} \quad \Rightarrow \quad \text{Separable: } 2 \cdot \mathcal{O}(k) \text{ ops/pixel}$$

resulting in lower runtime and higher effective MPix/s.

Metric	Value
Average Time (ms)	1.7036
Min Time (ms)	1.7009
Max Time (ms)	1.7070
Throughput (MPix/s)	39391.71
Ops per Pixel	121

Table 7: Run results for Variant 4: Filter/Kernel Separation.

8.1 NCU Reports:

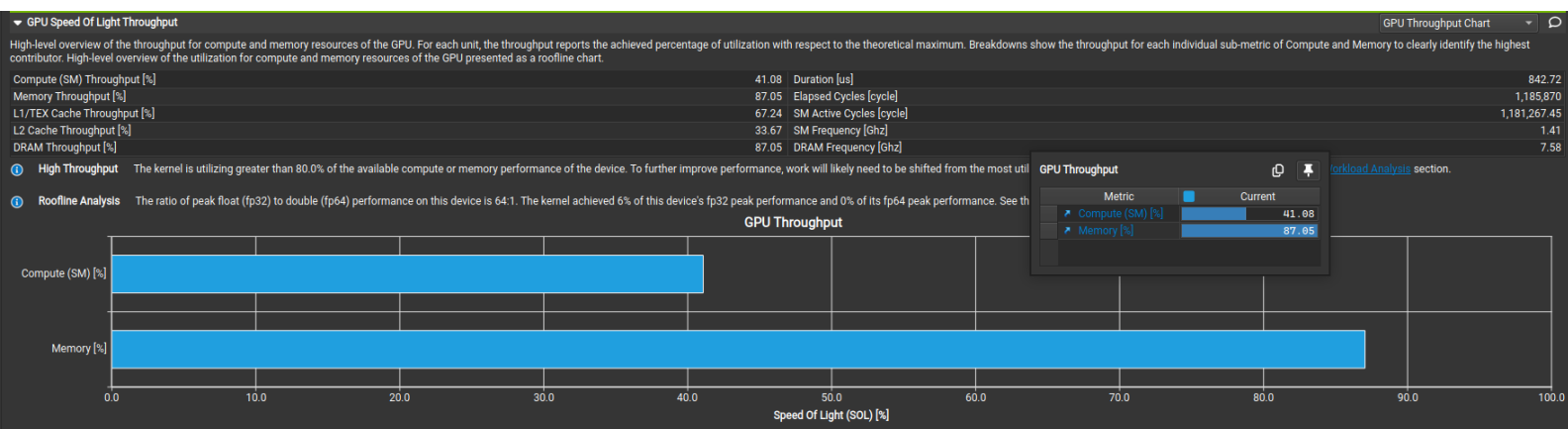


Figure 17: Variant 4 Row kernel

The compute goes down since the algorithmic cost per output pixel decreases from $\mathcal{O}(k^2)$ to $\mathcal{O}(2k)$, i.e., linear in the kernel size instead of quadratic.

Memory is still stressed since the compute does not dominate.

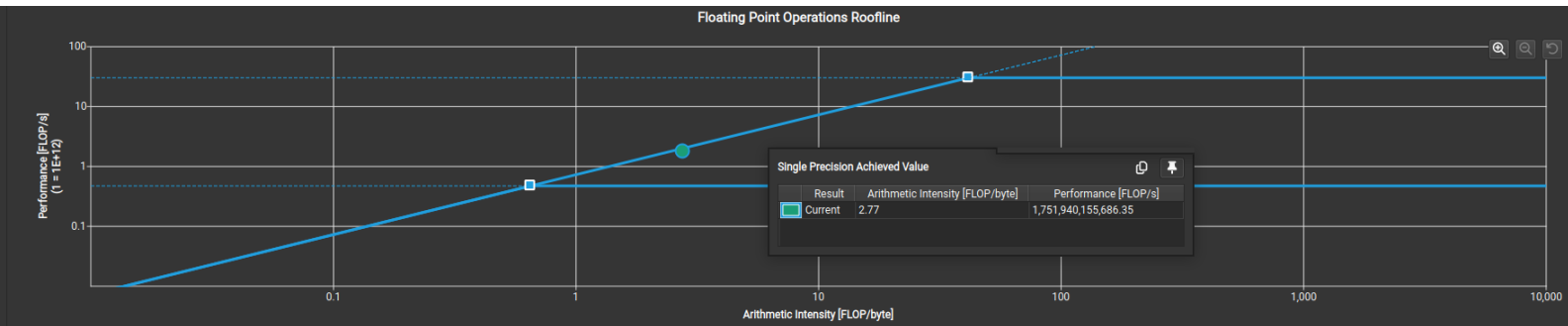


Figure 18: Variant 4 Row kernel

The Arithmetic intensity is very low now. The FLOP/s is very high due to the algorithmic optimization.

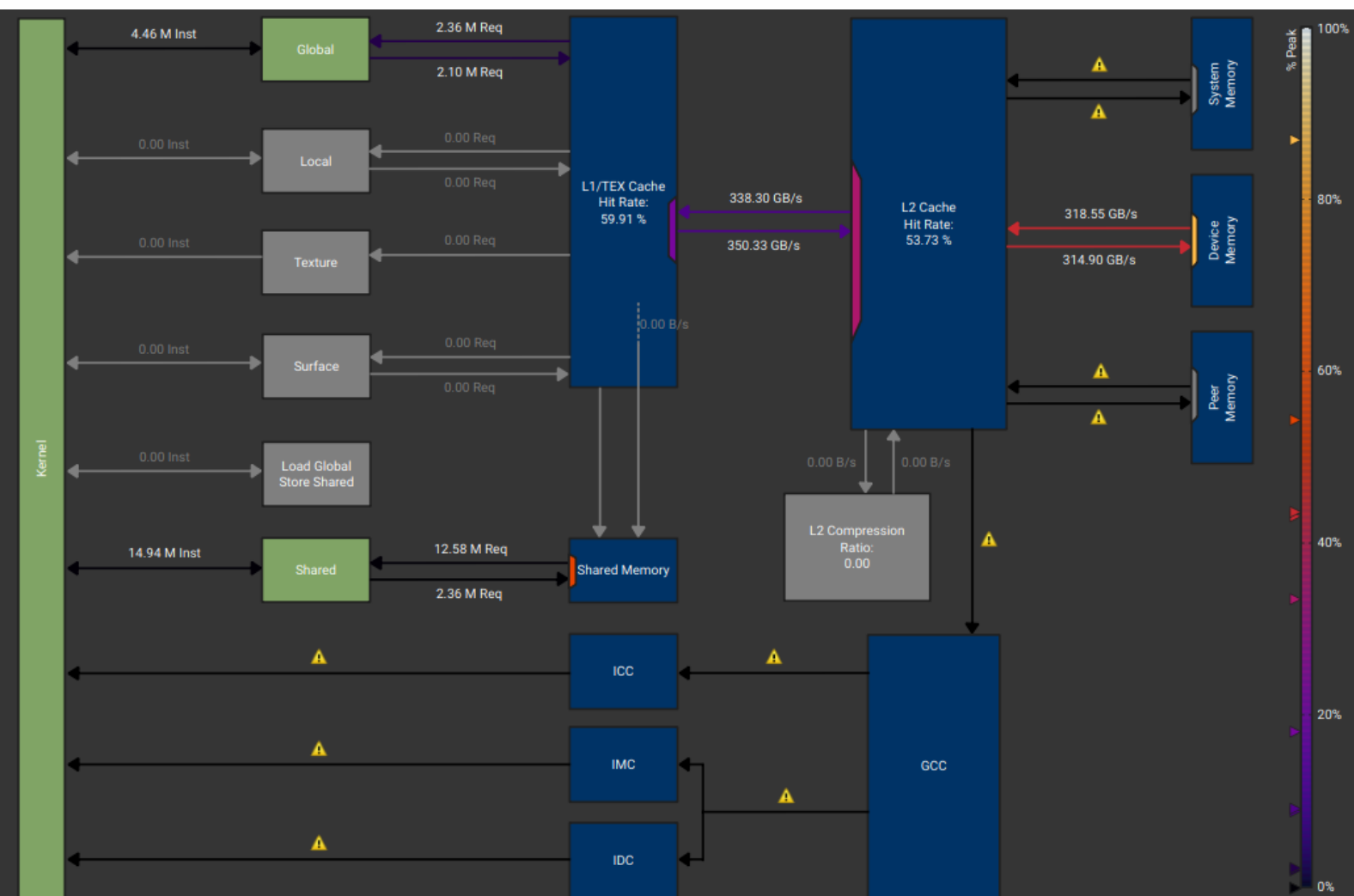


Figure 19: Variant 4 Row kernel

The L1 and L2 caches have lower hit rates since compute is now less complex and with register reuse the memory is stressed. All levels of the memory hierarchy see very high throughputs due to coalesced accesses.

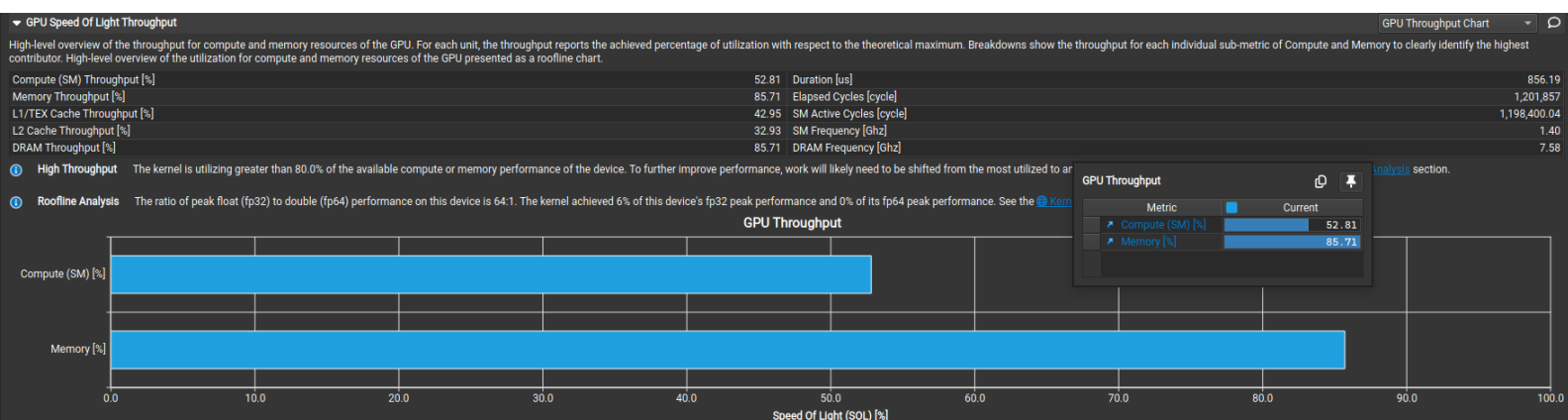


Figure 20: Variant 4 Col kernel

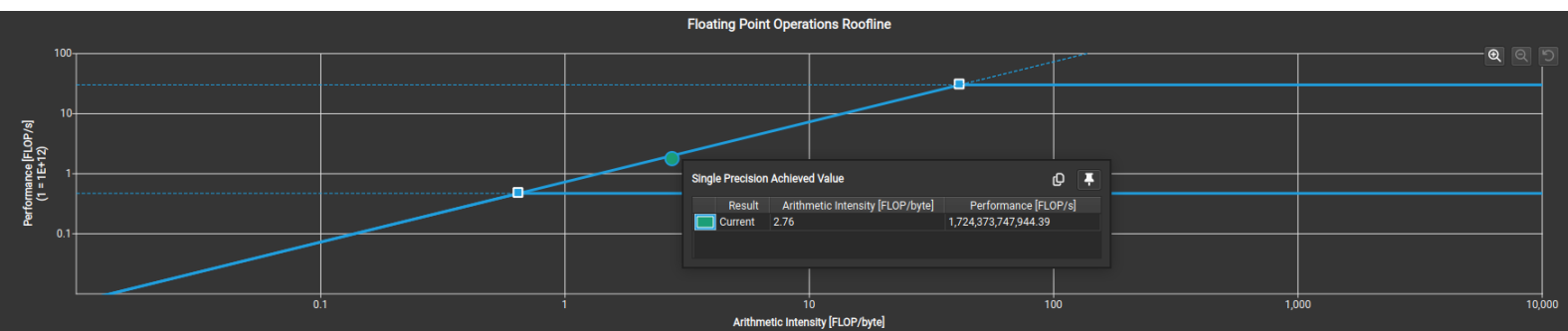


Figure 21: Variant 4 Col kernel

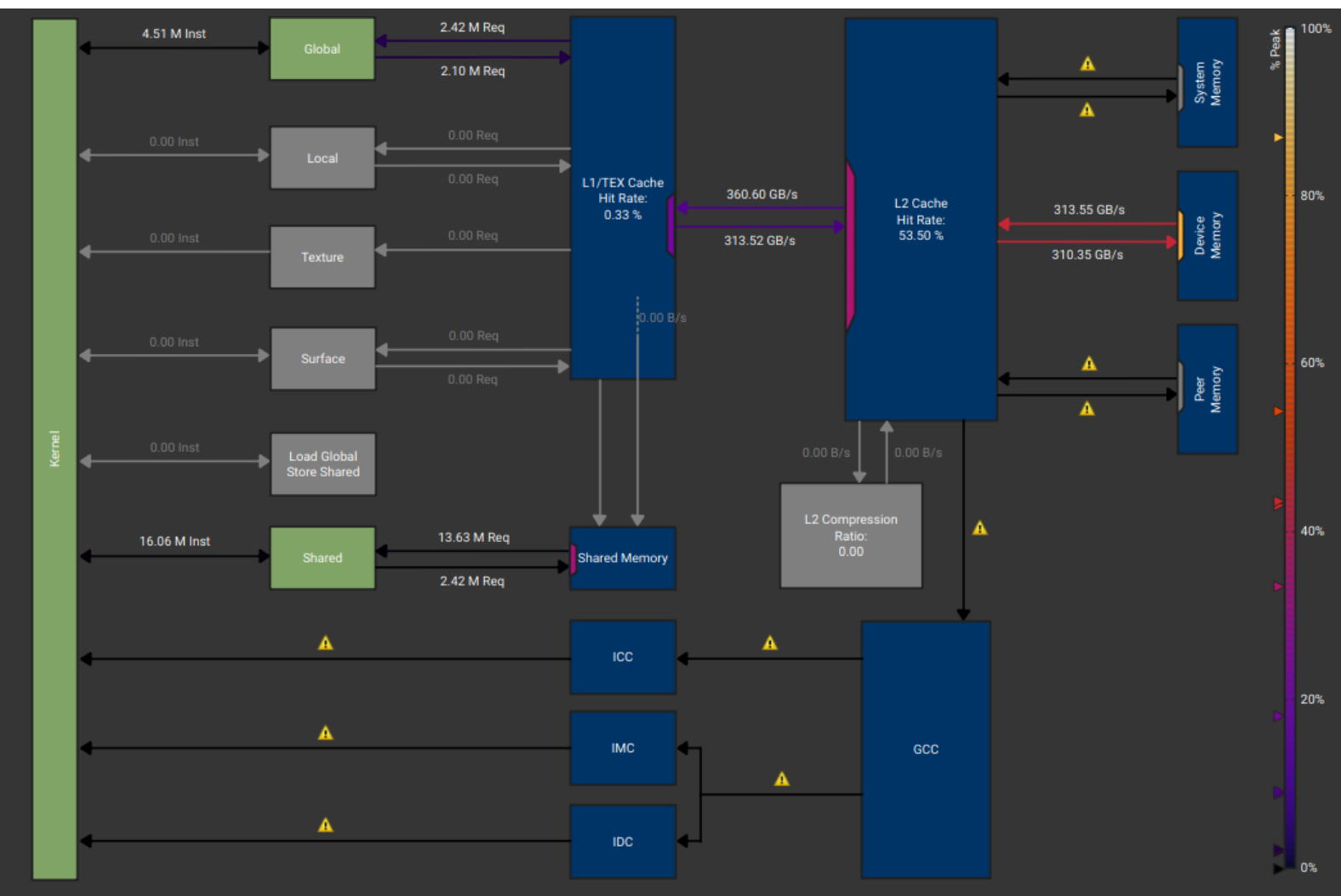


Figure 22: Variant 4 Col kernel

8.1.1 Major Factors:

This variant exploits kernel separability, eliminating a large portion of redundant multiply-accumulate operations.

The effect is significant: runtime drops to 1.70 ms compared to 2.05 ms in Variant 3 — roughly a 17% speedup. Global throughput rises to 39.39 GPix/s.

Even though the arithmetic intensity decreases (from 30.42 to 2.77 FLOP/byte) due to fewer operations per byte loaded, the achieved compute performance remains high at 1.75 TFLOP/s, because reduced compute pressure allows memory-side throughput to dominate efficiently. SM utilization also improves, indicating the kernel becomes less computation-bound and better aligned with GPU execution hardware.

Overall, separating the convolution into horizontal and vertical passes shifts the algorithm toward a better memory-to-compute balance, leading to the highest real-world performance among all variants.

9 Variants Comparison:

Table 8: Kernel Performance Comparison Across Variants (Averages)

Metric	Baseline	Variant1	Variant2	Variant3	V4-Col	V4-Row
Duration	37.59 ms	9.05 ms	4.81 ms	2.05 ms	0.858 ms	0.843 ms
Executed IPC Active	0.67	2.94	3.46	3.15	2.12	1.53
SM Busy	16.81%	73.47%	86.52%	78.85%	52.91%	38.35%
Achieved Occupancy	89.89%	92.10%	96.83%	97.28%	96.56%	—
Memory Throughput (GB/s)	14.22	59.04	111.04	260.19	622.68	—
Executed Instructions	2.99B	3.15B	1.97B	0.76B	0.21B	0.15B
Registers / Thread	40	31	33	40	40	40
Shared Mem per Block	0 B	8.19 KB	3.02 KB	9.94 KB	4.74 KB	—

Overall, the baseline version is strongly memory-bound due to uncoalesced global loads and almost no on-chip data reuse, resulting in the highest execution time and the lowest throughput.

Variant 1 alleviates this by restructuring thread layout to improve coalescing, which raises memory throughput but still relies heavily on hardware-managed caching.

Variant 2 eliminates most redundant global accesses by staging tiles in shared memory and moving the filter to constant memory, shifting execution toward a compute-balanced regime with significantly reduced stalls.

Variant 3 further pushes locality by reusing data directly in registers and increasing instruction-level parallelism, which cuts memory traffic per output and nearly doubles throughput again.

Finally, Variant 4 applies algorithmic optimization by exploiting filter separability, reducing arithmetic complexity from $\mathcal{O}(k^2)$ to $\mathcal{O}(2k)$ and achieving the lowest runtime overall. Each step reduces pressure on the slowest memory levels, allowing more sustained progress on the SMs; the cumulative effect is a nearly $22\times$ speedup from baseline to the fastest separable implementation.

Table 9: Runtime, Speedup and Compute Characteristics Comparison

Variant	Runtime (ms)	Speedup vs Baseline	GFLOP/s (2D-Equivalent)	Remarks
Baseline (Naive)	37.59	1.00×	430.91	Memory-bound
Variant 1: Global Coalescing	9.05	4.15×	1790.44	Partially coalesced
Variant 2: Shared + Constant	4.81	7.81×	3377.28	On-chip reuse
Variant 3: Register Tiling	2.05	18.33×	7876.20	High ILP + reuse
Variant 4: Separable	1.70	22.11×	—	Lower arithmetic cost

The separable filter implementation does not perform a full $k \times k$ convolution and therefore its FLOP/s values are not directly comparable to the other variants. While its compute throughput appears lower, the total arithmetic workload is drastically reduced from $\mathcal{O}(k^2)$ to $\mathcal{O}(2k)$, which is the fundamental reason for its runtime advantage rather than architectural tuning alone.